Lab Assignment 6

In [63]:
```
# importing libraries
import pandas as pd
```

In [64]:
```
# loading dataset
df=pd.read_csv("iris.csv")
```

In [65]:
```
# viewing the data
df.head()
```

Out[65]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [66]:
```
# getting information about dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

In [67]:
```
# cheking for null values
df.isnull().sum()
```

Out[67]:
```
sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

In [68]:
```
# dividing data into dependent and independent variable
x=df.drop("species",axis=1) # independent variable
x
```

Out[68]:

	sepal_length	sepal_width	petal_length	petal_width
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

150 rows × 4 columns

In [69]:

```
y=df["species"] # dependent variable
y
```

Out[69]:

```
0         setosa
1         setosa
2         setosa
3         setosa
4         setosa
         ...
145    virginica
146    virginica
147    virginica
148    virginica
149    virginica
Name: species, Length: 150, dtype: object
```

In [70]:

```
# encoding species column
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df['species_encod']=le.fit_transform(df['species'])
df.head()
```

Out[70]:

	sepal_length	sepal_width	petal_length	petal_width	species	species_encod
0	5.1	3.5	1.4	0.2	setosa	0
1	4.9	3.0	1.4	0.2	setosa	0
2	4.7	3.2	1.3	0.2	setosa	0
3	4.6	3.1	1.5	0.2	setosa	0
4	5.0	3.6	1.4	0.2	setosa	0

In [71]:
```
# droping species column(categorical column)
df=df.drop("species",axis=1)
df.head()
```

Out[71]:

	sepal_length	sepal_width	petal_length	petal_width	species_encod
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

In [72]:
```
print(le.classes_)
```

```
['setosa' 'versicolor' 'virginica']
```

In [73]:
```
# train test split
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,train_size=0.8)
```

In [74]:
```
# bulding model(naive bayes)
from sklearn.naive_bayes import GaussianNB
model=GaussianNB()
model.fit(x_train,y_train)
```

Out[74]:

```
▾ GaussianNB
▶ Parameters
```

In [75]:
```
import warnings
warnings.filterwarnings("ignore")
# predicting the value (single value)
predict_value=model.predict([[5.1,3.5,1.4,0.2]])
print(predict_value)
```

```
['setosa']
```

In [76]:
```
# predicting y
y_predict=model.predict(x_test)
```

In [77]:
```
# finding score
model.score(x_test,y_test)
```

Out[77]:
```
0.9666666666666667
```

In [78]:
```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, r
cm = confusion_matrix(y_test, y_predict)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[12  0  0]
 [ 0  9  0]
 [ 0  1  8]]
```

In [79]:
```
import numpy as np
# 10. Class-wise TP, FP, FN, TN
total = np.sum(cm)
print("\nClass-wise TP, FP, FN, TN:\n")
for i in range(len(cm)):
    TP = cm[i, i]
    FN = np.sum(cm[i, :])-TP
    FP = np.sum(cm[:, i])-TP
    TN = total-(TP + FP + FN)
    print(f"{le.classes_[i]}:")
print(f"TP={TP}, FP={FP}, FN={FN}, TN={TN}\n")
# 11. Overall Metrics
accuracy = accuracy_score(y_test, y_predict)
precision = precision_score(y_test, y_predict, average='macro')
recall = recall_score(y_test, y_predict, average='macro')
f1 = f1_score(y_test, y_predict, average='macro')
print("Overall Metrics:")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Class-wise TP, FP, FN, TN:

setosa:
versicolor:
virginica:
TP=8, FP=0, FN=1, TN=21

Overall Metrics:
Accuracy : 0.9666666666666667
Precision: 0.9666666666666667
Recall   : 0.9629629629629629
F1 Score : 0.9628482972136222
```